

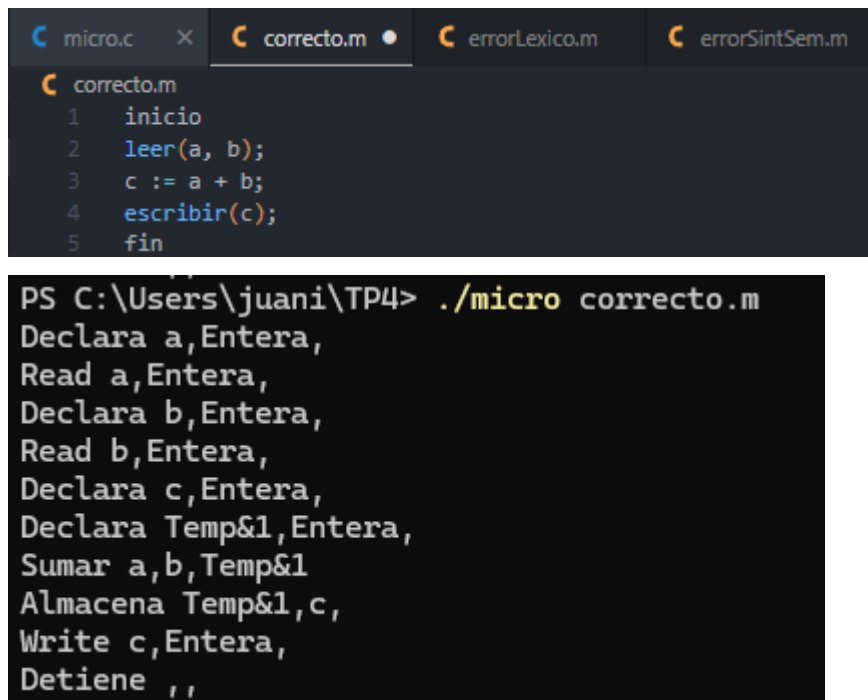
TP 4 - Grupo 8

Análisis del compilador de micro realizado en C

Integrantes:

- Ignacio Farid Juri
- Gustavo Ekiel
- Juan Ignacio Galzerano
- Felipe Carosella
- Juan Cruz Marotti

Primer ejemplo (correcto):



```
micro.c x correcto.m errorLexico.m errorSintSem.m
correcto.m
1 inicio
2 leer(a, b);
3 c := a + b;
4 escribir(c);
5 fin

PS C:\Users\juani\TP4> ./micro correcto.m
Declara a,Entera,
Read a,Entera,
Declara b,Entera,
Read b,Entera,
Declara c,Entera,
Declara Temp&1,Entera,
Sumar a,b,Temp&1
Almacena Temp&1,c,
Write c,Entera,
Detiene , ,
```

En este ejemplo que no hay errores de ningún tipo, se evidencia el correcto funcionamiento del compilador.

Inicialmente, observamos que inicialmente se declaran y se leen las variables “a” y “b”, debido a que en nuestro código llamamos a la función “leer(a, b);”

Luego se declara “c”, que a diferencia de las “2” anteriores, esta se declara por que está siendo igualada a la suma de “a” y “b”.

Durante la traducción, se declara una variable intermedia, “Temp&1”, para almacenar la suma de “a” y “b” y así almacenar es valor en “c”. Esto se realiza para separar la evaluación de la expresión de el almacenamiento del resultado.

Por último, se escribe “c” debido al llamado de la función “escribir(c)”, provocando la impresión del valor de “c”.

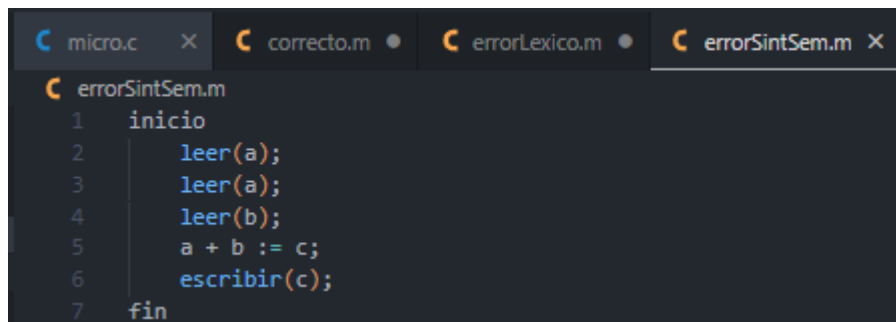
Segundo ejemplo (error léxico):

```
micro.c  correcto.m  errorLexico.m  errorSintSem.m
errorLexico.m
1  inicio
2  leer(%)a);
3  fin
4
```

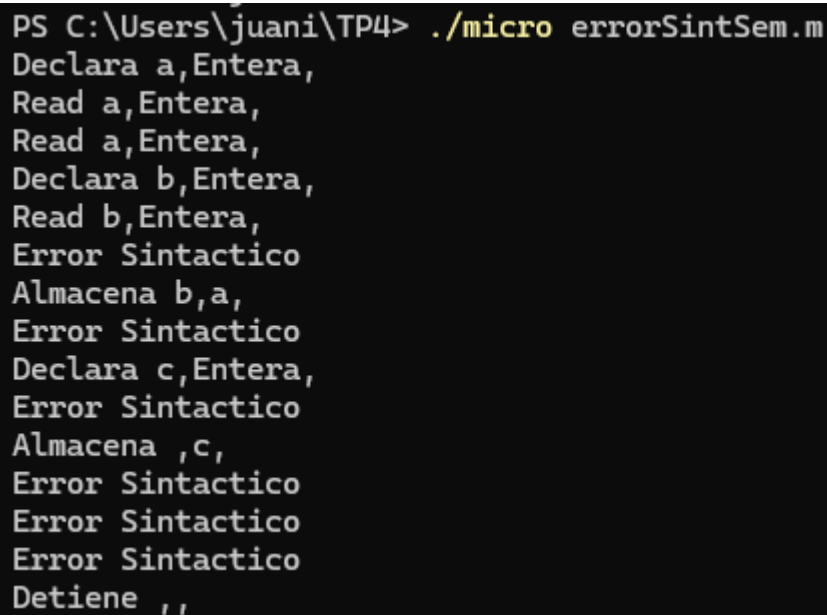
```
PS C:\Users\juani\TP4> ./micro errorLexico.m
Error Lexico
Error Sintactico
Declara %,Entera,
Read %,Entera,
Error Sintactico
Error Sintactico
Error Sintactico
Error Sintactico
Detiene ,,
```

En el segundo ejemplo escribimos un código de tal forma de que a la hora de leer el contenido de leer(), el analizador léxico no pueda declarar la variable "%a". Ya que el "%" no forma parte del lenguaje propuesto para las variables. Después de marcar este error léxico el compilador micro intenta seguir avanzando pero de forma errónea declarando "%" como entera saltando la "a". Luego de esto se computan sucesivos errores sintácticos gracias a que la sentencia quedo declarada erróneamente haciendo que no se comprendan el orden de los siguientes tokens.

Tercer ejemplo (error sintactico y semantico):



```
errorSintSem.m
1 inicio
2     leer(a);
3     leer(a);
4     leer(b);
5     a + b := c;
6     escribir(c);
7 fin
```



```
PS C:\Users\juani\TP4> ./micro errorSintSem.m
Declara a,Entera,
Read a,Entera,
Read a,Entera,
Declara b,Entera,
Read b,Entera,
Error Sintactico
Almacena b,a,
Error Sintactico
Declara c,Entera,
Error Sintactico
Almacena ,c,
Error Sintactico
Error Sintactico
Error Sintactico
Detiene ,,
```

Por último, este ejemplo muestra al principio cómo se lee dos veces la variable *a*. Esto constituye un error semántico, ya que si bien la sintaxis es válida, no tiene sentido lógico repetir la lectura de la misma variable sin necesidad.

Luego, al momento de realizar la asignación, la estructura utilizada es incorrecta. En este caso, se produce un error sintáctico. Es decir, el valor a asignar debe ubicarse a la derecha del operador “:=”, mientras que la variable que recibe el valor debe estar a la izquierda. En el ejemplo, la variable “*c*” está en el lugar equivocado, lo que genera el error de sintaxis.

En la consola se puede ver como no hay un solo error pero eso es gracias a lo explicado en el ejemplo anterior. Osea que, el compilador fuerza el avance y análisis del texto haciendo que siga pisando errores por malas declaraciones.